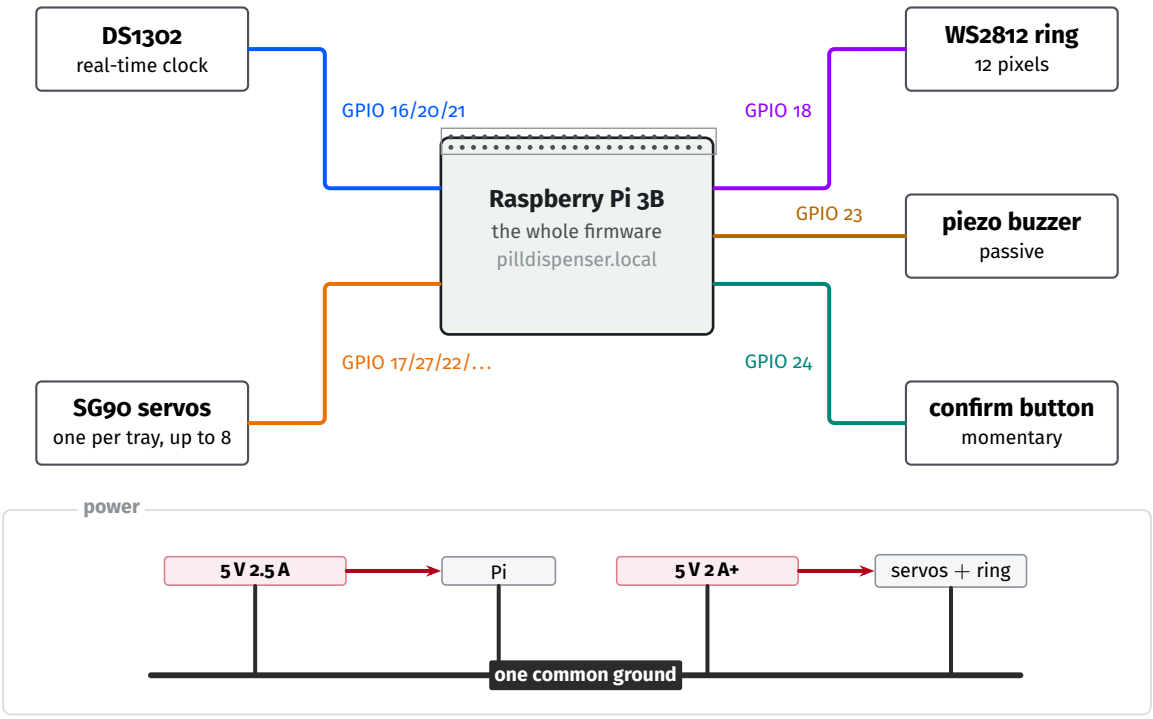
Building the dispenser

A bare Raspberry Pi 3B to a machine that drops a pill and chirps about it

SnapCare · Team H1 · 16th e-ICON World Contest



Eleven stages. Each one adds a single subsystem
and ends with the command that proves it works.

Matches `snapcare_dispenser/config.py`. Where the two disagree, that file is right.

Generated from `docs/assembly-guide.tex`.

Contents

1 The build at a glance 3

2 Bill of materials 3

3 The header, and every pin this build touches 5

4 Stage 0 — the Pi, before anything is wired to it 7

5 Stage 1 — the DS1302 7

6 Stage 2 — the confirm button 9

7 Stage 3 — the buzzer 9

8 Stage 4 — the LED ring 10

9 Stage 5 — the servo power rail, before any servo 11

10 Stage 6 — one servo 11

11 Stage 7 — the remaining servos 12

12 Stage 8 — mechanical fitting and trim 13

13 Stage 9 — cloud sync, if you want it 14

14 Stage 10 — the label 16

15 The whole pin map, in one place 17

16 Power budget 17

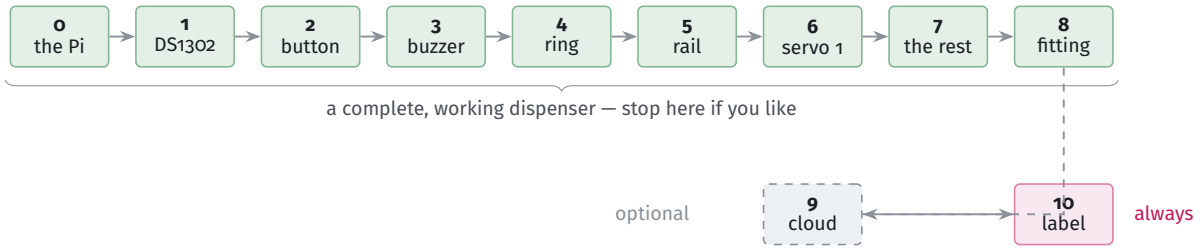
17 Commissioning 18

18 When it does not work 20

Read this first. Assemble in the order given and test after every stage. Each stage adds one subsystem and ends with a command that proves it. Wiring the whole thing and then powering it on means debugging five subsystems at once — and on this device three of the five fail *silently*. A servo on the wrong pin, a ring that never lights, and an RTC with a dead cell all look exactly the same from the outside: a machine sitting there doing nothing.

1 The build at a glance

Nothing in stages 0 to 8 depends on there being a network beyond the house.



	Subsystem	Where it lands	What it is for
	DS1302	GPIO 16/20/21 + 3.3 V	knows the time after a power cut
	WS2812 ring	GPIO 18, through 330 Ω	green / blue / amber / violet
	piezo buzzer	GPIO 23, through 100 Ω	the sound that fetches a person from another room
	confirm button	GPIO 24 to GND	internal pull-up, 20 ms glitch filter
	SG90 servos	GPIO 17, 27, 22, …	one per tray, signal only — never power

2 Bill of materials

	Qty	Part	Notes
	1	Raspberry Pi 3B	Any Pi with the 40-pin header works; the pin map assumes a 3B
	1	microSD card, 8 GB+	Raspberry Pi OS **Bookworm or newer** — the firmware drives WiFi through NetworkManager
5V	1	Pi power supply, 5 V 2.5 A	The official one. Servos do **not** come off this
	1 per tray	SG90 micro servo	Fit as many as you want hoppers, up to eight. The firmware accepts all eight, so a servo added later needs no change
5V	1	5 V supply, 2 A or better	Separate, for the servos and the ring
	1	1000 µF electrolytic, 10 V+	Across the servo rail, physically close to the servos
	1	WS2812 / NeoPixel ring, 12 px	16 and 24 also work — set `RING_PIXELS`
	1	330 Ω resistor	In the ring's data line, at the **ring** end
	1	Passive piezo buzzer	Passive, not an active beeper — see below

	Qty	Part	Notes
	1	100 Ω resistor	In series with the buzzer
	1	Momentary push button	Any normally-open button. No external resistor needed
	1	DS1302 RTC module	The 5-pin board with a coin-cell holder. Read the accuracy note below
	—	Jumper wire, terminal blocks	Dupont leads are fine for the bench; solder or screw terminals for anything that gets carried
	—	a relay, if you want cloud sync	*Not a part.* One container, deployed once for all your devices — see `../cloud_relay/`. Skip it and the dispenser is LAN-only

"Passive" buzzer

The firmware makes the tone itself — pigpio drives the pin as a 2.7 kHz square wave (`indicators.Buzzer.beep`). An *active* buzzer contains its own oscillator and sounds at its own frequency the moment it sees a voltage, so it will still make a noise here, just not the one that was chosen.



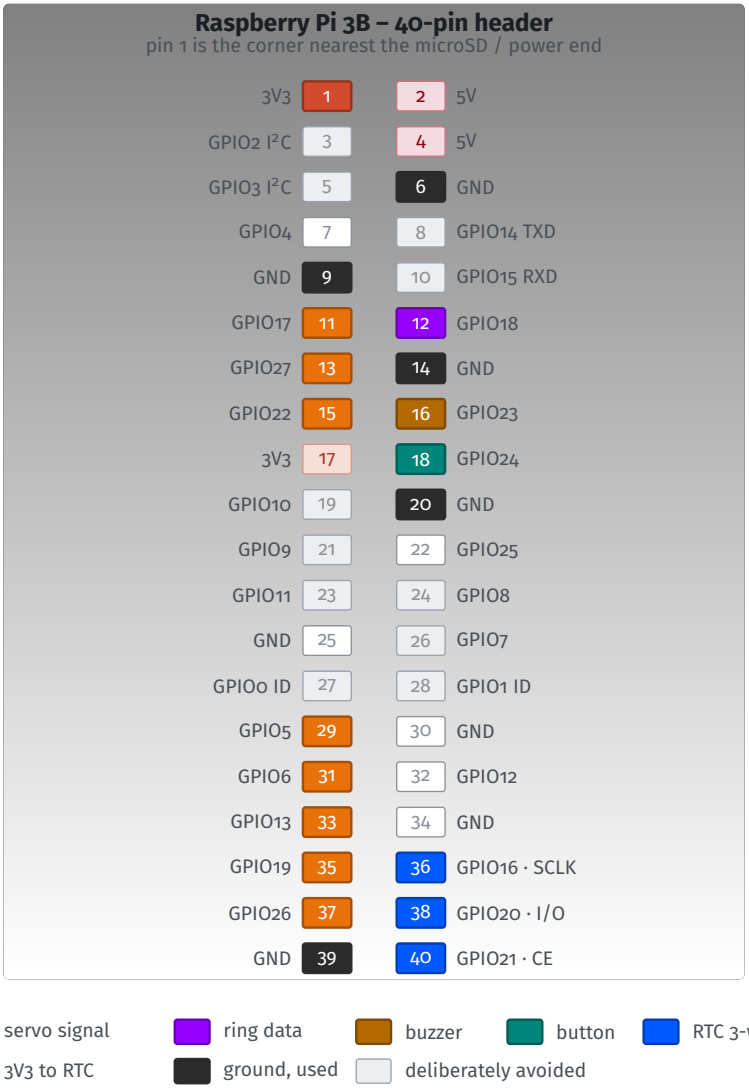
The DS1302 keeps worse time than you would expect

Its 32.768 kHz crystal is uncompensated, so a typical module drifts a few minutes a month, and more as the room warms and cools. That is good enough for the job it has here — carrying the time across a power cut until the Pi can reach an NTP server again — and it is *not* good enough as the only time source on a device that never sees a network. Commission this build on a network and leave it able to re-sync.

Fit the cell before you set the time, because what the chip remembers with the power off is the whole point of it. Most modules ship with a holder marked LIR2032; an ordinary non-rechargeable CR2032 is the safer choice and works, because the DS1302's built-in trickle charger stays switched off here.

3 The header, and every pin this build touches

Keep this page open while you wire. Everything is BCM numbering, which is what pigpio and RPi.GPIO both use; the small number inside each pad is the *physical* pin, which is what you actually count on the board.



The pins that were avoided, and what each costs

Pin	Why it is not used
GPIO 0, 1	HAT ID EEPROM. Driving them is how a HAT you plug in later fails to be detected
GPIO 2, 3	Permanent 1.8 kΩ pull-ups — a servo line idling high slams the horn at power-on. Harmless for the RTC's clock and data lines, which live here now
GPIO 14, 15	The serial console: the one way back into a Pi whose network setup you just broke
GPIO 7–11	SPI. Free today, gone the moment anyone adds an SPI display

The eight tray pins

GPIO 17, 27, 22, 5, 6, 13, 19 and 26, in tray order. All eight are checked for clashes and against the pins that cost hardware, and the firmware accepts all eight out of the box — so however many hoppers this machine has today, the next one is **a plug**: no config edit, no restart with a different number.

If you would rather the device *refuse* a schedule for a tray that is not physically there, say how many are fitted:

```
sudo systemctl edit snapcare-dispenser     # add:
      Environment=SNAPCARE_TRAYS=2
sudo systemctl restart snapcare-dispenser
```

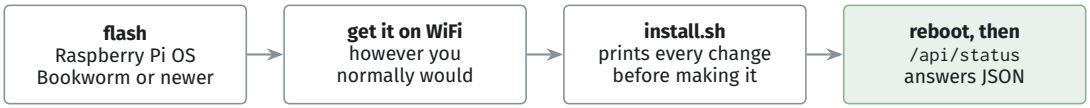
Set it in the unit rather than on a command line. Running

a second copy of the firmware by hand on a commissioned device fails anyway — port 80 is already taken by the service — and an environment variable typed at a shell does not survive the next reboot, which is the moment it stops being true.

`tests/test_servo.py` asserts all of this, and that no peripheral shares a pin with a servo.

4 Stage 0 — the Pi, before anything is wired to it

Do this stage first because it is the only one that can be done comfortably with the Pi on a desk and a keyboard attached. Once the servos are on it, every mistake costs a power-down.




```
git clone <this repo> && cd h1-SnapCare/pill_dispenser/rpi_firmware
sudo ./service/install.sh
sudo reboot
```

The installer is safe to re-run. It installs `python3-lgpio`, sets the hostname to `pilldispenser` and publishes it over avahi, writes the captive-portal DNS rule, installs `python3-cryptography` for cloud sync, and installs the systemd unit. The DS1302 needs no kernel overlay and no bus switched on: the firmware drives its three pins directly.

pigpio is gone, and that is expected. It was preinstalled on Raspberry Pi OS up to and including Bookworm and disappeared at the Trixie rebase — Debian has never packaged the daemon, only the client, and the 64-bit images bootstrap from Debian. So `sudo apt install pigpio` fails on a current 64-bit card and there is no way to make it work.

The installer tries it anyway, because a Bookworm card still has it and DMA timing is genuinely better, and carries on when it is not there. `python3-lgpio` is the one that matters: it is pre-installed on every Raspberry Pi OS Lite image since 2023 and it is what will drive your pins. `/api/status` says which one came up, and `--selftest` will not sign off a build that landed on neither.

Prove it

```
systemctl status snapcare-dispenser
curl -s http://localhost/api/status
```

You should get JSON. The `gpio` object will list what is missing, which right now is everything — correct, nothing is wired yet. The `cloud` object will say `unpaired`, which is also correct: no relay has been named.

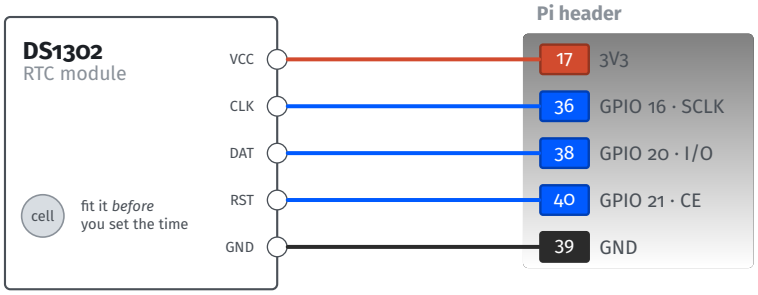
The clock will most likely say `trusted` already, with `source: ntp`, because Stage 0 just put this Pi on WiFi. That is not the RTC working — nothing is wired to read yet. The coin cell is proved by the power-cut test at the end and by nothing before it.

5 Stage 1 — the DS1302

Power off first — every time

Not “put it to sleep”. Unplug it. Every wiring stage in this guide starts that way, and this reminder is not repeated after this page.

Five wires. Three of them land on the last three pins of the header’s even-numbered row — 36, 38 and 40, at the far end from the microSD slot — with ground on pin 39 right beside them. The 3.3 V comes from pin 17.



3.3 V, not 5 V. The chip runs at either, but its `I/O` line *actively drives* back at whatever it is powered from, straight into a GPIO that is not 5 V tolerant.

Go by the silkscreen. The five terminals are in a different order on almost every board sold. Match the names, not the positions drawn here.

Physical pin numbers are in the pads. For where each one sits on the board, see the header map on page 5.

This is not a bus, and there is no kernel driver. The DS1302 is three plain GPIOs shifted by hand in software, so `i2cdetect`, `/dev/rtc0` and `hwclock` have nothing to say about it — the firmware is the only thing that can read this chip, and `/api/status` is where you look.

GPIO 2 and 3 carry permanent 1.8 kΩ pull-ups. They are harmless on a clock and a data line, which is why the RTC gets them; `CE` is put on GPIO 4, which has none. That same pull-up is why GPIO 2 and 3 are kept out of the servo map: a servo line that idles high before the software starts slams the horn to one end at power-on, with a loaded tray on it.

Prove it

```
curl -s http://localhost/api/status     # the "clock" object: trusted, source, rtc
curl -X POST http://localhost/api/clock \
     -H 'Content-Type: application/json' -d '{"iso":"2026-07-29T09:15:00"}'
```

Then prove the only thing the chip is there for. Unplug the Pi, leave it a minute, and bring it back with the network unplugged: `trusted` must still be true and `source` must read `rtc`. A module that reads back the time you just set, but loses it across a power cut, has a dead cell or a cell holder that is not making contact.

This chip cannot answer the question the firmware asks

Worth understanding before trusting it, because it is the one place the DS1302 is a real step down from the DS3231 the Arduino build used.

A DS3231 has an *oscillator stop flag*: the hardware sets it whenever the oscillator has stopped and it stays set until software clears it. That answers the only question worth asking — *did this chip lose time at some point, so that everything it has said since is invented?*

The DS1302 has no such bit. Its clock-halt flag is a control, not a status: nothing in the hardware ever sets it, and its power-on state is documented as undefined. A clear flag means only “running right now”, which is equally true of a chip that sat unpowered for a month and resumed counting from whatever was left in its registers.

So the firmware reconstructs the evidence instead. The DS1302 has 31 bytes of RAM in the *same battery-backed domain as the oscillator* — the coin cell holds both or neither. Setting the clock writes a token there, and on boot that token is present if and only if the cell has held continuously since. It is firmware convention rather than a hardware guarantee, and both of its failure modes fall towards *not* trusting the clock, which is the safe direction. `snapcare_dispenser/ds1302.py` is the whole argument.

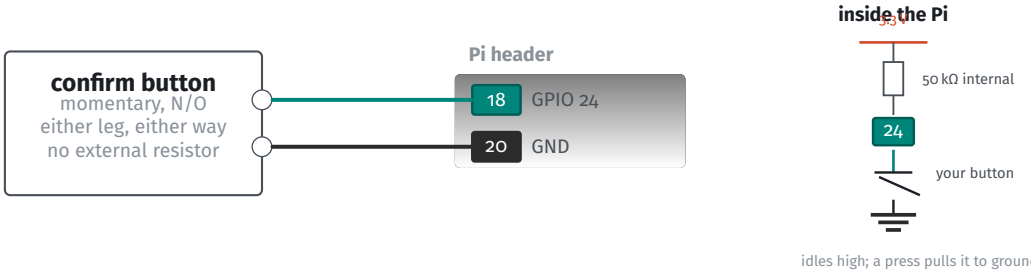
There is no kernel driver, so nothing sets the clock but this firmware

On a DS3231 the kernel reads the chip and sets the system clock before userspace starts. The DS1302 has no such path on a Pi at all, so the firmware reads it itself at startup — which happens later, after systemd has already timestamped its own boot. That is the cost of the part.

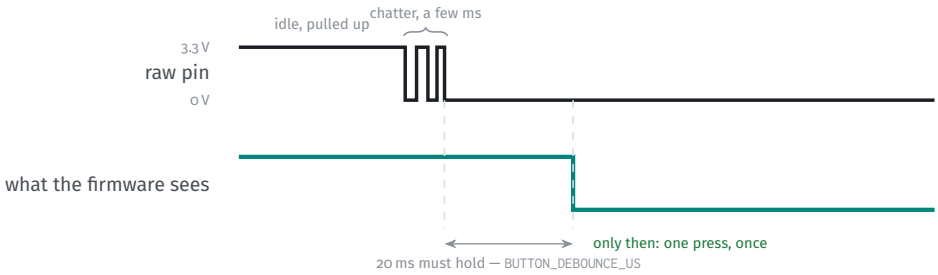
Two things follow. `i2cdetect`, `/dev/rtc0` and `hwclock` have nothing to say about this chip, so `/api/status` is the only place to look; and `fake-hwclock` is no longer competing with a kernel RTC, so leaving it installed is harmless.

6 Stage 2 — the confirm button

Two wires, nothing else.



Why there is no debounce capacitor. A real switch does not close once; it chatters for a few milliseconds. Pigpio's glitch filter discards any edge that does not then hold for 20 ms (`BUTTON_DEBOUNCE_US`), *before* a callback is ever queued. Undebounced, one press records several doses.



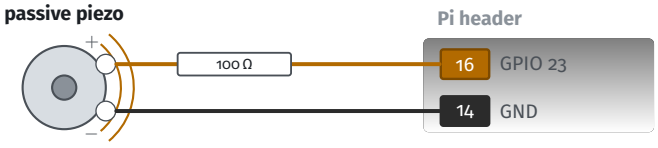
Prove it

You need a dose waiting to confirm, so this is easiest tested at the end of Stage 3 — but you can check the firmware claimed the pin now:

```
journalctl -u snapcare-dispenser | grep -i button
```

It should report the `pigpio` backend. `poll` means pigpiod is not running, and `none` means neither is available.

7 Stage 3 — the buzzer



Any free GPIO would do — the tone is software-timed PWM from pigpio, so it does not compete with the ring for a hardware PWM channel.

The tone is **2.7 kHz**, 0.12 s, repeating every 20 s while a dose is waiting.

Prove it

The buzzer is proved by the self-test, which sounds it deliberately and asks what you heard:

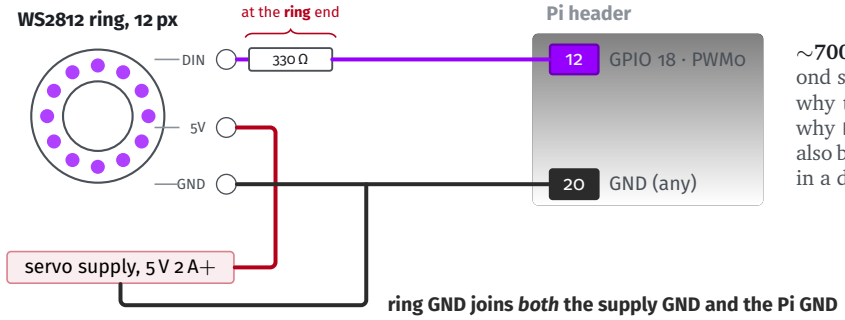
```
sudo systemctl stop snapcare-dispenser
sudo python3 -m snapcare_dispenser --selftest
```

What you are listening for is a clear *tone*, not a click. A click means an active buzzer with its own oscillator, which makes a noise at whatever frequency it likes rather than the one the firmware asked for — fit a passive piezo element instead.

`POST /api/drop` deliberately does **not** chirp. A manual drop is a nurse checking a hopper, not a dose somebody has to be fetched for, so it moves the servo and stays silent. The repeating twenty-second chirp belongs to a *scheduled* dose, and the way to see that is to schedule one a minute out.

8 Stage 4 — the LED ring

This is the first stage that touches the servo supply. The ring's 5 V does *not* come from the Pi.



~700 mA at full white. That is a second servo's worth of current, which is why the ring is on the servo rail and why `RING_BRIGHTNESS` is 60 of 255 — also because full brightness is dazzling in a dark bedroom at 3 a.m.

GPIO 18 is not a free choice

`rpi_ws281x` generates the WS2812's 800 kHz waveform with the PWM or PCM peripheral and the DMA controller, and **GPIO 18, 12, 21 and 10 are the only pins wired to them.** Anything else is a ring that never lights and an error message about DMA. GPIO 18 is PWM0 and is not in the servo map. A test asserts this (`tests/test_servo.py:181`).

The 330 Ω is not folklore. It damps the reflection on the first data edge, which is what makes the first pixel latch a wrong colour on a long lead. It has to be at the ring end to do that.

Prove it

```
sudo systemctl restart snapcare-dispenser
curl -s http://localhost/api/status     # "indicators" should now list the ring
curl -X POST http://localhost/api/schedule \
     -H 'Content-Type: application/json' \
     -d '{"payload":"SC1*W*1:0800-127-030"}'   # violet: schedule accepted
```

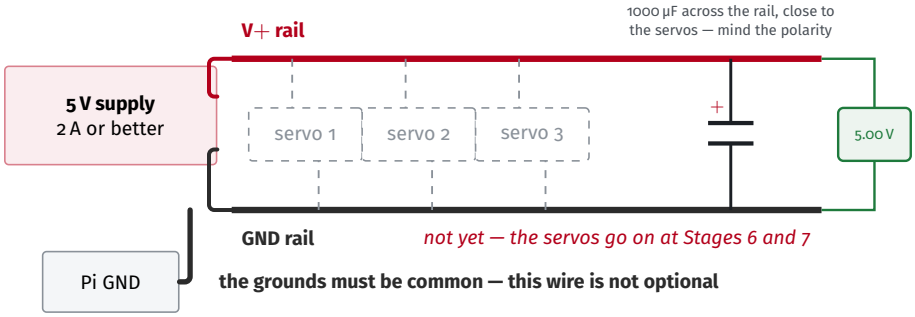
If the ring stays dark, check `journalctl` — a ring that throws is dropped and logged rather than retried forever, and the status page reports what is actually fitted rather than pretending.

The colour vocabulary you have just fitted. Never text, because the person the machine is for cannot be assumed to read.



9 Stage 5 — the servo power rail, before any servo

Wire and measure the rail with **no servos attached and no signal wires to the Pi**. This is the stage that protects the SD card.



Do not run the servos from the Pi's 5 V pin

One SG90 stalls at around 700 mA. The failure is not a blown regulator; it is a brown-out, and **a Pi that browns out mid-write corrupts an SD card** — where the file being written is the one that records whether a dose has already been given.

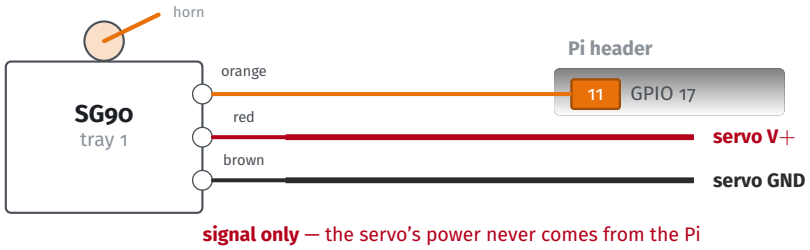
The common ground is what makes the signal mean anything. Without it the servos twitch or ignore commands entirely, which reads as a firmware fault and is not one. The bulk capacitor absorbs the current step when a servo starts, which is what otherwise dips the rail.

Prove it

Meter across the rail. **5 V ±0.25 V, steady.** Nothing else is connected yet, so anything else is the supply or the wiring, not the firmware.

10 Stage 6 — one servo

Do one, entirely, before the other two.



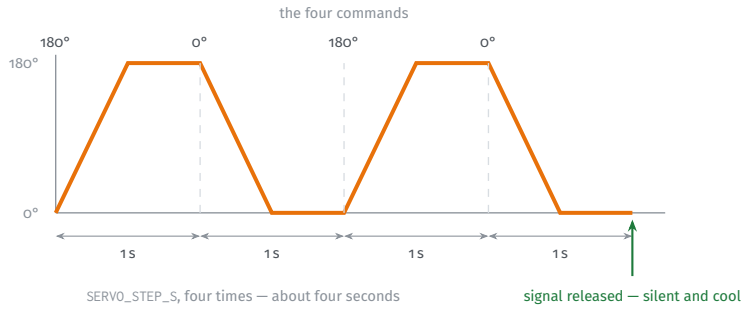
Prove it

```
sudo systemctl restart snapcare-dispenser
curl -X POST http://localhost/api/drop \
     -H 'Content-Type: application/json' -d '{"tray":1}'
```

The tray rests at 0°. A drop is four commands from there — 180°, 0°, 180°, 0°— one second apart, so about four seconds in total. Then the signal to that pin is **cut** rather than held — so

> the servo should go silent and cool between doses.

Why the stroke runs twice. One tip does not reliably clear the gate: a pill can sit on the lip and ride back with the horn, which reads as a dose given and is not one. The second round trip is what shakes it through.



Check which driver you got. `servoBackend` in the status, with `servoSteady` saying whether its timing can be trusted:

```
curl -s http://localhost/api/status | grep -i servo
```

Backend	Verdict	What it means
`pigpio`	what you want	Pulses timed by the DMA controller, immune to CPU load
`rpigpio`	jittery fallback	Pulses timed from a Python thread; every kernel preemption is a visible twitch and an audible buzz. Reports `servoSteady: false`. Fix with `sudo systemctl enable --now pigpiod`
`mock`	nothing moves	No servo library present at all

11 Stage 7 — the remaining servos

Tray	GPIO	Header pin
1	17	11
2	27	13
3	22	15
4	5	29
5	6	31
6	13	33
7	19	35
8	26	37

Work down this table for as many hoppers as the machine has. Nothing in the firmware says how many that is — all eight are accepted by default, and every pin here is already checked for clashes and against the pins that cost hardware. A hopper added next month is a plug and nothing else.

Wire them one at a time and prove each before the next. The trade-off that the default buys is below.

What “all eight by default” costs, and how to buy it back

An SG90 reports nothing, so there is no way for the firmware to tell a fitted hopper from an empty pin. Accepting all eight means a schedule naming a tray this machine does not physically

have is **accepted** — and that dose is then never dispensed and reported by nothing. No jam, no error, a status page that looks perfectly healthy.

On a machine that will not grow, say how many are fitted and the firmware refuses such a schedule by name, while somebody is still looking at the screen that sent it:

```
sudo systemctl edit snapcare-dispenser     # add: Environment=SNAPCARE_TRAYS=2
```

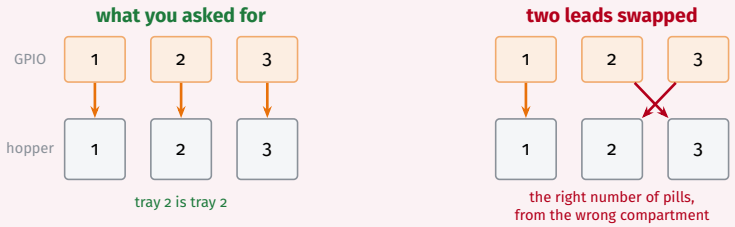
Prove it

Prove each one *individually*, and watch which hopper actually moves. Set `TRAYS` to the number you fitted — the sleep is longer than one drop, so the moves stay separated and you can see which is which:

```
TRAYS=2
for t in $(seq 1 $TRAYS); do
  curl -X POST http://localhost/api/drop \
       -H 'Content-Type: application/json' -d "{\"tray\":$t}"
  sleep 6
done
```

This is the one worth doing carefully

Two trays on one pin gives the right number of pills from the wrong compartment, and **nobody notices that until the wrong medication has been taken for a week.** The firmware refuses a duplicate pin at startup, but it cannot check that the wire you plugged into GPIO 27 goes to the hopper you are calling tray 2.



Only one servo is ever driven at a time, enforced by a lock — three doses owed at the same minute still move one horn at a time. Eight SG90s starting together is a current step that reboots the Pi with a compartment open.

12 Stage 8 — mechanical fitting and trim

Fit the hoppers with the servos at their **resting 0°**, not wherever the horn happened to stop. Park each one there first, then mount the horn:

```
sudo python3 -m snapcare_dispenser --drop 1     # sweeps, and ends at rest
```



If a tray does not quite reach its stop, or buzzes at the end of the stroke because it is being driven past it, trim *that tray alone*:

```
SERVO_TRIM = {3: (520, 2450)}    # config.py -- overrides only tray 3
```

Buzzing at the end of a stroke is the one to act on

It is the servo fighting a mechanical stop it has already reached, and it is what burns one out over a few weeks.

The other knob, if a full hopper is slower than an empty one:

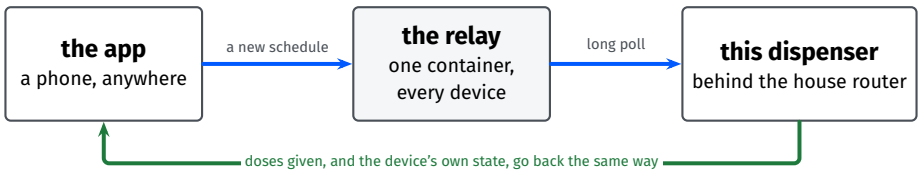
```
SERVO_STEP_S = 1.0     # seconds between one command in the stroke and the next
```

This is the whole timing model. Nothing on an SG90 reports position, so the firmware is simply waiting out the clock, and the interval must cover the *slowest* case — a cold servo under the weight of a full tray, about half a second — and still leave the hopper time to settle before the horn reverses. Raising it lengthens the drop by four times whatever you add.

13 Stage 9 — cloud sync, if you want it

Skip this stage entirely and everything still works: the schedule arrives from the app over the home WiFi or off a photographed paper sheet, and the device runs it from the SD card against the DS1302. That is the whole machine, and it is a reasonable way to ship one.

What this stage adds is that a schedule edited in the app reaches the device **without anybody being in the house** — which is the normal case, because the person managing a prescription is usually not the person taking it.



the relay is *not* a part in the bill of materials:
one container, deployed once, serving every dispenser you build

See `../cloud_relay/README.md`. Then name it here:

```
curl -X POST http://localhost/api/cloud \
     -H 'Content-Type: application/json' \
     -d '{"relayUrl":"https://snapcare-relay.fly.dev"}'
```

or type it into the device's own console, or set `SNAPCARE_RELAY_URL` in the systemd unit if you are building more than one.

Prove it

```
curl -s http://localhost/api/status     # the "cloud" object
```

`state` should reach `connected` within a few seconds, with `lastError` empty and `reportedRev` at 1 or more — that last one is the device having published its own state, which is what creates its mailbox.

Then prove the direction that matters, from the relay's demo app: open `https://<your relay>/lab`, pair with this device's code, and send it a schedule. It should appear in `payload` here within a second or two.

If it does not:

`state`	What it means
`connecting`	The first poll is in flight. This is what you see first, for a second or two
`unpaired`	No relay URL. The POST above did not take, or was sent to the wrong host
`unsupported`	`python3-cryptography` is missing. `sudo apt install python3-cryptography`
`retrying`	The relay is not answering. `lastError` says why — usually DNS or a firewall
`offline`	It was reachable and has not been for fifteen minutes
`off`	Switched off — `--no-cloud`, or `SNAPCARE_CLOUD=0`

The test nobody does: unplug the internet, not the device

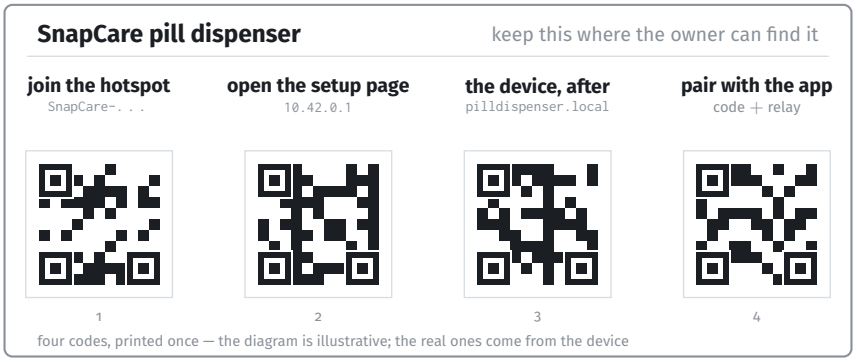
Pull the router's WAN cable, leave the dispenser on its LAN, and check that a scheduled dose still comes out on time. It must. A build that has just gained a cloud feature is exactly when a dependency on it gets introduced by accident, and the symptom is a machine that works perfectly in every test anyone runs at a desk.

14 Stage 10 — the label

The setup code is printed once and never changes. It is derived, not generated: HMAC-SHA256 over the SoC serial in `/proc/cpuinfo`, so it survives a reimage and differs per board. The pairing code on it comes from the same derivation, so it is permanent in the same way.

```
sudo python3 -m snapcare_dispenser --show-qr      # scannable, in the terminal
```

or print the proper one from `http://pilldispenser.local/label`.



The pairing code is a credential

Anyone who can read it can send this device a schedule, in the same way anyone who can read the WiFi password can join the hotspot. Put the label where the owner can find it and a visitor cannot — inside a battery door, or on the underside.

The one part of the label that is not permanent

The pairing QR encodes *your relay's address* alongside the code, because that is what makes a phone camera able to open it. Move to a different relay and that QR points at the old one. The code printed beside it in text still works — it is derived from the board, not from the relay — so the fix is to reprint, not to re-pair. If you expect to move relays, print the label with no relay configured: the QR then carries the code alone, which the app can read and a plain camera cannot.

15 The whole pin map, in one place

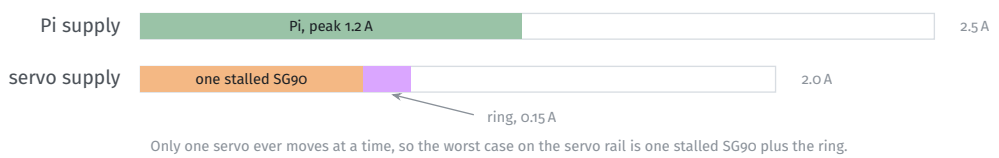
	Signal	GPIO (BCM)	Header pin	Notes
	Servo tray 1	17	11	signal only — **not** power
	Servo tray 2	27	13	
	Servo tray 3	22	15	
	Servo trays 4–8	5, 6, 13, 19, 26	29, 31, 33, 35, 37	same status as the first three; fit a hopper and it works
	WS2812 ring data	18	12	through 330 Ω at the ring; must be 18/12/21/10
	Piezo buzzer	23	16	through 100 Ω
	Confirm button	24	18	other leg to GND; internal pull-up
	DS1302 SCLK	16	36	bit-banged; no bus, no kernel driver
	DS1302 I/O	20	38	bidirectional — 3.3 V only
	DS1302 CE	21	40	
	DS1302 VCC	—	17 (3.3 V)	not 5 V
	Ground	—	6, 9, 14, 20, 25, 30, 34, 39	pin 9 to the RTC; at least one to the servo supply GND

Run `pinout` on the Pi for the full header. Everything above is BCM numbering, which is what pigpio and RPi.GPIO both use.

16 Power budget

Two supplies, and the split is the point.

Rail	Load	Draw
Pi supply (5 V 2.5 A)	Pi 3B with WiFi active	~0.4 A idle, ~1.2 A peak
Servo supply (5 V 2 A+)	one SG90, moving	~0.2 A typical, ~0.7 A stalled
Servo supply	WS2812 ring, 12 px at brightness 60	~0.15 A (≈0.7 A at full white)



Only one servo ever moves at a time, so the worst case on the servo rail is one stalled SG90 plus the ring.

Figures for the servo and the ring are the ones the firmware was designed around; the Pi's are the usual published ranges. Meter your own rail if you change the ring size. Only one servo ever moves at a time, so the tray count does not move these figures.

17 Commissioning

Run this before the device goes anywhere near real medication.

```
sudo systemctl stop snapcare-dispenser
sudo python3 -m snapcare_dispenser --selftest
```

It checks what can be checked, asks about what only eyes and ears can settle, and writes a report to `/var/lib/snapcare/commissioning.json`. It opens trays, so it says so and waits for you to agree first.

Why this is not a paper checklist any more. The list this replaced was a good list with a structural problem: most of its claims were not things a person can actually observe.

"Servos silent and cool a few seconds after a drop" is a real failure with a real consequence and it is unanswerable three stages later, so the self-test waits three seconds after the last stroke and asks then. "Every tray you fitted moves" is ticked truthfully by a machine whose trays 1 and 2 are wired to each other's pins — both moved — so the self-test asks *which hopper moved*, by number. And nothing on paper can tell a button that fires once per press from one that fires four times, which is the entire reason the debounce exists, so the self-test counts the edges.

The rule it follows throughout: never ask about something it did not just make happen.

It reports in three states, and the third is the one worth reading:

`ok`	Checked and correct
`FAIL`	Checked and wrong, with the fix beside it
`warn`	True but not blocking — an lgpio-timed servo, a tray count left at the default
`skip`	**Nobody looked.** "Everything passed" and "everything anybody looked at passed" are different sentences, and this is the difference

Five things it cannot do, which are the five that matter most. Each needs the power, the network or the calendar interfered with, and each is invisible in any test run at a desk:

Test	What it catches
The power cut	Pull the power at 08:30, after the 08:00 dose, and restore it. No second dose, and the log survives. This is exactly the case that dispenses twice without persisted state
The coin cell	Unplug the Pi, leave it a minute, bring it back with the network unplugged. `clock.trusted` must still be true and `clock.source` must read `rtc`. Nothing on a bench with WiFi proves this, because NTP vouches first and the RTC is never consulted
The wrong password	Type the WiFi password wrong on purpose. The setup network must come back within a few seconds and the page you re-open must say the password was not accepted
The internet cut	Pull the router's WAN cable and leave the dispenser on the LAN. A scheduled dose must still come out on time. This is the one that catches a dependency on the relay introduced by accident, because everything passes at a desk
The repeated revision	Send the same schedule revision twice; the second must be ignored. The relay refuses it, so this is the only way to see the *device's* own check — which is what stands between a rolled-back relay and last month's prescription being applied as new

And two more if the machine will be left with somebody:

- ☐ Change the router's password, reboot the device, wait: the setup hotspot appears on its own rather than the device sitting there unreachable
- ☐ Power the router and the device off together and bring only the device back for a few minutes first. It raises its hotspot, then finds the router by itself once that returns — no hands

18 When it does not work

Symptom	Look at
Servo twitches or ignores commands	Common ground. The Pi's GND and the servo supply GND must be joined
Servo hums between doses	You are on the `rpigpio` fallback — `sudo apt install python3-lgpio`. `gpio.servo.timing` in `/api/status` says which you have: `dma`, `threaded` or `python`
Nothing moves and no error appears	No GPIO library at all. The firmware now refuses to dispense rather than logging doses that never came out — `gpio.missing` says so. `sudo apt install python3-lgpio`
`apt install pigpio` says there is no such package	Correct, on Raspberry Pi OS Trixie. It is not coming back; lgpio is the driver now
Servo buzzes at the end of a stroke	Being driven past its mechanical stop. Trim it in `SERVO_TRIM` before it burns out
Pi reboots when a servo moves	Servos are on the Pi's 5 V pin, or the rail has no bulk capacitor
Ring never lights, DMA error in the log	Data line is not on GPIO 18/12/21/10
First pixel is the wrong colour	330 Ω missing, or at the Pi end instead of the ring end. If it persists, the ring's 5 V logic threshold ($0.7 \times V_{DD} = 3.5$ V) is above the Pi's 3.3 V output — shorten the lead, or drop the ring's supply to ∼4.5 V
Ring flickers only when a servo moves	pigpio and the ring contending for PWM. pigpiod's `-t` selects its clock peripheral: `-t 1` is PCM, which is the default and the one that leaves PWM0 free for the ring. `-t 0` selects PWM and is the setting that *causes* this
Buzzer makes a tone but not *this* tone	It is an active buzzer with its own oscillator. Use a passive element
One press logs several doses	Debounce — check `journalctl` says the `pigpio` button backend, not `poll`
Tray stays shut, nothing dispenses	`device.blockedReason` in `/api/status` says which of the two reasons it is: an untrusted clock, or no GPIO library. They need opposite fixes
Clock right at shutdown, wrong at boot	The cell is dead or not seated. `clock.rtc.detail` distinguishes "never been set" from "the cell has been flat since"
Clock drifts a few minutes a month	Expected — the DS1302's crystal is uncompensated. Leave the device able to reach NTP
`pilldispenser.local` does not resolve	avahi, and the hostname. The web app's `DEFAULT_HOST` is that exact string
Ring is cyan and stays cyan	The device is on its own setup hotspot waiting to be put on a network. Join `SnapCare-. . .` and the page opens itself
Setup page says "did not hand out an address"	The password was right and DHCP was not. Usually a router that needs restarting, or a MAC filter
The setup hotspot keeps disappearing for half a minute	That is the unattended retry finding a saved network worth trying. It stops once one works; raise `HOTSPOT_RETRY_S` if it is in the way of a bench session
A schedule POST answers 421	The `Host` header is not a name this device answers to. Use `pilldispenser.local` or the IP; `SNAPCARE_EXTRA_HOSTS` adds a router's own domain
Setup page never leaves "waiting…"	The phone has not gone back to the house WiFi. Switch it over; the page follows. If the device's own network reappeared, the password was refused

Symptom	Look at
A schedule is refused by tray number	This build was told how many trays it has. Raise `SNAPCARE_TRAYS`, or drop it to use all eight
A dose for a tray that was never fitted never comes out, and nothing complains	All eight are accepted by default and an SG90 reports nothing. Set `SNAPCARE_TRAYS` to the number really fitted
Cloud `state: unsupported`	`sudo apt install python3-cryptography`. Everything else still works meanwhile
Cloud `state: retrying`, DNS in `lastError`	The Pi has no working resolver. It joined the WiFi but got no DNS, or the router blocks it
Cloud connects, but no schedule ever arrives	The app is paired to a different code. Compare `cloud.deviceId` here with the one the app shows — they must match exactly
The app shows the device as never seen	Nothing has written to the mailbox. Check `reportedRev` here is 1 or more
A schedule sent from the app is refused	`cloud.lastError` says why, in the same words the LAN API would use. Usually a tray that is not fitted
Cloud fine, but doses take minutes to appear	`SNAPCARE_CLOUD_IDLE_S` is set. That is the cheap mode working as asked

Nothing here needs the hardware to reproduce the *logic* side of it:

```
make            # the full suite, no board, no servos, no network
make run        # the whole firmware on a laptop, mock servos, port 8099
make -C .. demo # the same, plus a relay and the demo app on localhost:8088
```

`make run` is the real firmware with the servo driver swapped for one that only records what it was told, and `make demo` adds a relay and a browser page that pairs with it — so the whole cloud path, including its failure cases, can be exercised without a Raspberry Pi anywhere.